MANUAL DE DESPLIEGUE

API NestJS + App Next.js

Levantamiento del entorno, carga de datos y verificación del sistema

Versión 1.0 • 2025 • Uso interno / equipos técnicos

1. Introducción

Este manual describe el proceso completo para levantar el entorno de desarrollo o producción del sistema, compuesto por dos proyectos independientes que trabajan en conjunto:

- **API NestJS:** Backend REST que expone los servicios del sistema, conectado a una base de datos PostgreSQL administrada mediante Docker.
- **App Next.js:** Frontend web que consume la API y sirve la interfaz de usuario.

Además, se documenta el proceso de carga manual de datos iniciales en la base de datos mediante un script SQL, paso obligatorio para el correcto funcionamiento del sistema.

> ⚠ **IMPORTANTE:**
> Sigue el orden del manual estrictamente. La API debe estar funcionando y con los datos cargados antes de levantar el frontend.

2. Requisitos previos

Asegúrate de tener instalado y configurado lo siguiente antes de comenzar:

Requisito	Descripción / Valor
Node.js	v18.x o superior. Verificar con: node --version
npm / yarn	npm v9+ o yarn v1.22+. Verificar con: npm --version

Docker	Docker Engine en ejecución. Verificar con: docker ps
Docker Compose	v2.x. Verificar con: docker compose version
Git	Para clonar los repositorios. Verificar con: git --version
NestJS CLI	Instalado globalmente: npm install -g @nestjs/cli
Repositorios clonados	Ambos proyectos (API y App) descargados y en la ruta correcta
Archivo .env configurado	Variables de entorno correctas en cada proyecto (ver sección 3.1 y 4.1)
Script SQL disponible	El archivo ./scripts/carga_calendario_2026.sql debe existir en la raiz del proyecto API

> ℹ **NOTA:**
> Si Docker no está corriendo, inicia el servicio antes de continuar. En Linux: sudo systemctl start docker. En macOS/Windows: abre Docker Desktop.

3. Resumen del proceso

El proceso completo se divide en dos grandes bloques, cada uno con sus propios pasos:

BLOQUE A — API NestJS

A1	**Configurar variables de entorno** Crear y revisar el archivo .env de la API
A2	**Instalar dependencias** Ejecutar npm install en el proyecto de la API
A3	**Levantar contenedores Docker** Iniciar PostgreSQL y servicios dependientes con Docker Compose
A4	**Ejecutar migraciones** Aplicar el esquema de base de datos
A5	**Cargar datos del calendario 2026** Insertar data inicial con el comando docker exec + script SQL
A6	**Iniciar la API** Levantar el servidor NestJS en modo desarrollo o producción

BLOQUE B — App Next.js

B1	**Configurar variables de entorno** Crear y revisar el archivo .env del frontend
B2	**Instalar dependencias** Ejecutar npm install en el proyecto Next.js
B3	**Iniciar la App** Levantar el servidor Next.js en modo desarrollo o producción

BLOQUE A — Configuración y arranque de la API NestJS

A1 — Configurar variables de entorno (API)

En la raíz del proyecto de la API, crea el archivo .env basándote en el archivo de ejemplo. Este archivo define la conexión a la base de datos y otros parámetros del servidor.

```
# Ir a la carpeta del proyecto API
$ cd /ruta/a/tu-proyecto-api

# Copiar el archivo de ejemplo
$ cp .env.example .env

# Editar el archivo con tu editor preferido
$ nano .env
# o: code .env    (si usas VS Code)
```

Las variables mínimas requeridas son:

Variable	Valor de ejemplo / Descripción
`DB_HOST`	localhost (o nombre del servicio Docker)
`DB_PORT`	5432
`DB_USERNAME`	postgres
`DB_PASSWORD`	tu_password_segura
`DB_NAME`	api_db
`PORT`	3000 (puerto donde escucha la API)
`NODE_ENV`	development o production

⚠ **IMPORTANTE:**

Nunca subas el archivo .env al repositorio. Verifica que esté incluido en .gitignore antes de hacer cualquier commit.

A2 — Instalar dependencias de la API

Instala todas las dependencias del proyecto NestJS:

```
# Asegúrate de estar en la carpeta raíz del proyecto API
$ cd /ruta/a/tu-proyecto-api

# Instalar dependencias
$ npm install

# Alternativa con yarn:
$ yarn install

# Verificar que no hay errores críticos
$ npm audit --audit-level=high
```

✓ **TIP:**
Si ves errores de peer dependencies, usa: npm install --legacy-peer-deps. Reporta cualquier vulnerabilidad crítica al equipo antes de continuar en producción.

A3 — Levantar contenedores Docker

La API depende de una base de datos PostgreSQL administrada por Docker. Levanta los contenedores con el siguiente comando desde la raíz del proyecto:

```
# Levantar todos los servicios definidos en docker-compose.yml en segundo plano
$ docker compose up -d

# Verificar que los contenedores están corriendo
$ docker ps

# Debes ver algo como:
# CONTAINER ID   IMAGE        STATUS         NAMES
# e98335222197   postgres:15  Up X seconds   api_db  (o similar)

# Ver los logs de PostgreSQL si necesitas verificar:
$ docker compose logs postgres
```

⌨ **COMANDO CLAVE:**

Anota el CONTAINER ID del contenedor PostgreSQL. Lo necesitarás en el Paso A5 para ejecutar el script SQL. En el ejemplo de este manual el ID es: e98335222197

```
# Obtener el ID del contenedor PostgreSQL en cualquier momento:
$ docker ps --filter name=postgres --format '{{.ID}}'
# o buscar por nombre de imagen:
$ docker ps --filter ancestor=postgres
```

A4 — Ejecutar migraciones de base de datos

Aplica las migraciones para crear el esquema y las tablas en la base de datos. Este paso solo es necesario la primera vez o cuando hay cambios en el esquema.

```
# Ejecutar las migraciones pendientes
$ npm run migration:run

# Alternativas según la configuración del proyecto:
$ npx typeorm migration:run -d src/data-source.ts
# o con npx ts-node:
$ npx ts-node -r tsconfig-paths/register node_modules/.bin/typeorm migration:run

# Verificar el estado de las migraciones:
$ npm run migration:show
```

ℹ NOTA:
Si el script migration:run no existe en tu package.json, consulta con el equipo cuál es el comando correcto para tu configuración de TypeORM o Prisma.

A5 — CARGA MANUAL DE DATOS: Calendario 2026

A5 — Cargar datos del calendario 2026

Este es un paso manual obligatorio. Debes ejecutar un script SQL dentro del contenedor Docker de PostgreSQL para insertar los datos del calendario 2026 en la base de datos.

⚠ IMPORTANTE:
Este paso debe ejecutarse SIEMPRE que se levante el entorno por primera vez, o si se resetea la base de datos. Sin estos datos, algunas funcionalidades del sistema no operarán correctamente.

A5.1 — Verificar que el script SQL existe

Antes de ejecutar el comando, confirma que el archivo SQL está en la ubicación correcta:

```
# Desde la raíz del proyecto API, verificar que el archivo existe:
$ ls -lh ./scripts/carga_calendario_2026.sql

# Debes ver una salida similar a:
# -rw-r--r-- 1 user group 45K Jan 10 09:00 ./scripts/carga_calendario_2026.sql

# Si no existe, contacta al equipo para obtener el archivo
```

A5.2 — Ejecutar el script SQL en el contenedor

Ejecuta el siguiente comando desde la raíz del proyecto de la API (donde se encuentra la carpeta ./scripts/):

> **COMANDO CLAVE:**
> Este es el comando exacto para la carga de datos. Ejecutalo tal como aparece, reemplazando el ID del contenedor si es necesario.

```
# COMANDO PRINCIPAL — Ejecutar desde la raíz del proyecto API:
$ docker exec -i e98335222197 psql -U postgres -d api_db -v ON_ERROR_STOP=1 <
./scripts/carga_calendario_2026.sql

# Desglose del comando:
#  docker exec -i          → Ejecuta comando en el contenedor en modo interactivo
#  e98335222197            → ID del contenedor PostgreSQL (verificar con docker
ps)
#  psql                    → Cliente de PostgreSQL
#  -U postgres             → Usuario de la base de datos
#  -d api_db               → Nombre de la base de datos destino
#  -v ON_ERROR_STOP=1      → Detiene la ejecución si hay un error en el SQL
#  < ./scripts/carga_...   → Redirige el archivo SQL como entrada al comando
```

A5.3 — Interpretación del resultado

Después de ejecutar el comando, observa la salida en pantalla:

Salida en pantalla	Significado
`INSERT 0 N (varios)`	Datos insertados correctamente. N = cantidad de filas.
`SET`	Variable de configuracion psql aplicada correctamente.
`Sin salida / silencio`	El script se ejecuto sin errores (comportamiento normal en psql).
`ERROR: duplicate key...`	Los datos ya estan cargados. No es necesario re-ejecutar.

`ERROR: ON_ERROR_STOP`	Hubo un error en el SQL. Ver mensaje de error para diagnostico.

A5.4 — Verificar la carga de datos

Confirma que los datos se insertaron correctamente consultando directamente la base de datos:

```
# Conectarse al contenedor e iniciar psql:
$ docker exec -it e98335222197 psql -U postgres -d api_db

# Dentro de psql, consultar los datos cargados:
api_db=# SELECT COUNT(*) FROM calendario;
api_db=# SELECT * FROM calendario LIMIT 5;

# Salir de psql:
api_db=# \q
```

> ✓ **TIP:**
> Si el ID del contenedor cambió (por ejemplo, después de un docker compose down/up), obten el nuevo ID con: docker ps | grep postgres

A6 — Iniciar la API NestJS

Con la base de datos lista y los datos cargados, ya puedes iniciar el servidor de la API:

Modo desarrollo (con hot-reload)

```
# Iniciar en modo watch (recompila al detectar cambios)
$ npm run start:dev

# La API estará disponible en:
# http://localhost:3000

# Documentación Swagger (si está configurada):
# http://localhost:3000/api
```

Modo producción

```
# Compilar el proyecto primero:
$ npm run build

# Iniciar en modo producción:
$ npm run start:prod
```

```
# Alternativa con PM2 para mayor estabilidad:
$ pm2 start dist/main.js --name api-nestjs
$ pm2 logs api-nestjs
```

✓ **TIP:**
Verifica que la API responde antes de levantar el frontend. Prueba: curl http://localhost:3000/health o abre el navegador en esa URL.

BLOQUE B — Configuración y arranque de la App Next.js

B1 — Configurar variables de entorno (App Next.js)

En la raíz del proyecto Next.js, configura las variables de entorno. Next.js usa el prefijo NEXT_PUBLIC_ para las variables accesibles desde el navegador.

```
# Ir a la carpeta del proyecto Next.js
$ cd /ruta/a/tu-proyecto-nextjs

# Copiar el archivo de ejemplo
$ cp .env.local.example .env.local

# Editar el archivo
$ nano .env.local
```

Variables mínimas requeridas en el frontend:

Variable	Valor de ejemplo / Descripción
`NEXT_PUBLIC_API_URL`	http://localhost:3000 (URL de la API NestJS)
`NEXT_PUBLIC_APP_ENV`	development o production
`NEXTAUTH_SECRET`	Clave secreta para sesiones (si usas NextAuth)
`NEXTAUTH_URL`	http://localhost:3001 (URL del frontend)

⚠ **IMPORTANTE:**
La variable NEXT_PUBLIC_API_URL debe apuntar exactamente a donde está corriendo la API NestJS. Si cambias el puerto de la API, actualiza esta variable y reinicia el frontend.

B2 — Instalar dependencias de la App Next.js

```
# Asegúrate de estar en la carpeta raíz del proyecto Next.js
$ cd /ruta/a/tu-proyecto-nextjs

# Instalar dependencias
$ npm install

# Alternativa con yarn:
$ yarn install
```

B3 — Iniciar la App Next.js

Modo desarrollo

```
# Iniciar en modo desarrollo (hot-reload activado)
$ npm run dev

# La app estará disponible en:
# http://localhost:3001  (o el puerto configurado)

# Next.js por defecto usa el puerto 3000.
# Si la API ya usa el 3000, especifica otro puerto:
$ PORT=3001 npm run dev
# En Windows (PowerShell):
$ $env:PORT=3001; npm run dev
```

Modo producción

```
# Compilar el proyecto:
$ npm run build

# Iniciar en modo producción:
$ npm run start

# Con PM2:
$ pm2 start npm --name app-nextjs -- start
$ pm2 logs app-nextjs
```

4. Verificación del sistema completo

Una vez levantados ambos servicios, realiza las siguientes comprobaciones para confirmar que el sistema funciona correctamente de extremo a extremo:

#	Verificación	Cómo comprobarlo	Resultado esperado
1	Contenedores Docker activos	docker ps	Contenedor postgres en estado Up
2	API responde	curl http://localhost:3000/health	Status 200 OK o JSON de estado
3	Datos del calendario cargados	Consulta SELECT COUNT(*) FROM calendario en psql	Numero mayor a 0
4	Swagger accesible (si aplica)	Abrir http://localhost:3000/api	Documentacion de endpoints visible
5	App Next.js cargada	Abrir http://localhost:3001	Pagina principal del sistema visible
6	Frontend conectado a API	Navegar una seccion que consuma datos	Datos reales del calendario aparecen

5. Detener los servicios

Para detener el entorno de forma ordenada, sigue estos pasos:

```
# 1. Detener la App Next.js:
#    Presiona Ctrl+C en la terminal donde corre el frontend
#    o si usas PM2:
$ pm2 stop app-nextjs

# 2. Detener la API NestJS:
#    Presiona Ctrl+C en la terminal donde corre la API
#    o si usas PM2:
$ pm2 stop api-nestjs

# 3. Detener los contenedores Docker (sin eliminar datos):
$ docker compose stop

# 4. Eliminar los contenedores (los datos en volumen se conservan):
$ docker compose down

# CUIDADO: Eliminar contenedores Y volúmenes (BORRA LA BASE DE DATOS):
$ docker compose down -v   # SOLO si quieres resetear todo
```

> ⚠ **IMPORTANTE:**
> Usa docker compose down -v SOLO si deseas resetear completamente la base de datos. Deberás volver a ejecutar las migraciones y el script de carga del calendario (Paso A5) desde cero.

6. Solución de problemas frecuentes

Problema	Causa probable	Solución
docker compose up falla	Puerto 5432 ya en uso por otro servicio	Detén PostgreSQL local: sudo systemctl stop postgresql o cambia el puerto en docker-compose.yml
Error: ECONNREFUSED al iniciar API	La BD no está lista o el DB_HOST es incorrecto	Verifica que docker ps muestra el contenedor en estado Up. Revisa DB_HOST en .env
docker exec: no such container	El ID del contenedor cambió tras reiniciar	Ejecuta docker ps, copia el nuevo ID y actualiza el comando del Paso A5
ERROR: duplicate key en el script SQL	Los datos del calendario ya estaban cargados	No es un error crítico. Puedes ignorarlo o limpiar la tabla antes de re-ejecutar
ON_ERROR_STOP: error en el SQL	El script encontró un error de SQL	Lee el mensaje de error completo. Puede ser que la tabla no exista (falta migración)
npm run start:dev no arranca	Falta variable de entorno o dependencia	Revisa que .env existe y tiene todas las variables. Ejecuta npm install nuevamente
Next.js: fetch failed a la API	NEXT_PUBLIC_API_URL apunta a URL incorrecta	Verifica la variable en .env.local. Recuerda reiniciar Next.js tras cambiar .env
Puerto 3000 en uso	Otro proceso ocupa el puerto	Cambia el puerto en .env (PORT=3001) o mata el proceso: lsof -ti:3000 | xargs kill

7. Glosario

Requisito	Descripción / Valor
NestJS	Framework de Node.js para construir APIs REST escalables, basado en TypeScript y arquitectura modular
Next.js	Framework de React para aplicaciones web con renderizado del lado del servidor (SSR) y generacion estatica (SSG)
Docker	Plataforma de contenedores que empaqueta aplicaciones con sus dependencias para ejecutarlas en cualquier entorno
Docker Compose	Herramienta para definir y ejecutar aplicaciones multi-contenedor mediante un archivo YAML

PostgreSQL	Sistema de gestión de base de datos relacional open source. Se ejecuta dentro del contenedor Docker
docker exec	Comando para ejecutar instrucciones dentro de un contenedor Docker en ejecución
psql	Cliente de línea de comandos de PostgreSQL para ejecutar consultas y scripts SQL
ON_ERROR_STOP	Parámetro de psql que detiene la ejecución del script SQL si ocurre cualquier error
Migraciones	Scripts que aplican cambios al esquema de la base de datos de forma controlada y versionada
.env	Archivo de configuración con variables de entorno. Nunca debe subirse al repositorio de código
PM2	Process Manager para Node.js. Mantiene las aplicaciones corriendo en producción y las reinicia si fallan
Hot-reload	Característica de los modos de desarrollo que recarga la aplicación automáticamente al detectar cambios en el código

8. Historial de versiones

Versión	Fecha	Autor	Cambios
v1.0	2025	Equipo Técnico	Versión inicial. Incluye flujo completo API NestJS + App Next.js + carga manual de datos calendario 2026